

REACT

Crear proyecto --> **npm create vite@latest**

Luego --> **npm install**

Ejecutar --> **npm run dev**

Dentro de **src** crear carpeta **components** para crear archivos en ella ejemplo **HeaderComponent.jsx**

Index.css da estilos a todo el proyecto

GUIA EXAMEN SPOTIFEX

1. SpotiMain.jsx

Componente principal

Componente que contiene todo

App

└─ SpotiMain

└─ ListadoCanciones

└─ FiltradoCanciones

└─ MasPopular

```
SpotifEx > src > components > SpotiMain.jsx > ...
1  import { useEffect, useState } from "react"
2  import ListadoCanciones from "../ListadoCanciones"
3  import FiltradoCanciones from "../FiltradoCanciones"
4  import MasPopular from "../MasPopular"
5
6  const SpotiMain = () => {
7    const [canciones, setCanciones] = useState([]) // crea variable de estado, empieza como array vacio
8
9    const cargarCanciones = async () => { // asincrona para obtener array
10      try {
11        // await: js termine de cargar para continuar
12        const datos = await fetch("../public/json/Spotify.json") // busca archivo
13        const datos = await respuesta.json() // convierte archivo en datos utilizables en Js
14        setCanciones(datos) // guarda las canciones en el estado
15      } catch (error) {
16        console.log("Error al cargar canciones:", error)
17      }
18    }
19
20    // Hace que la carga se ejecute una sola vez
21    useEffect(() => {
22      cargarCanciones()
23    }, [])
24
25    return(
26      <div>
27        <h1>SpotifEx</h1>
28        <MasPopular canciones={canciones}></MasPopular>
29        <ListadoCanciones canciones={canciones}></ListadoCanciones>
30        <FiltradoCanciones canciones={canciones}></FiltradoCanciones>
31      </div>
32    )
33  }
34
35
36
37  export default SpotiMain
```

2. APP.jsx

Simplemente llamar al componente, App.jsx es quien lo va a “mostrar”

```
SpotifEx > src > App.jsx > default
1  import { useState } from 'react'
2  import reactLogo from './assets/react.svg'
3  import viteLogo from './assets/vite.svg'
4  import heroImg from './assets/hero.png'
5  import './App.css'
6  import SpotiMain from './components/SpotiMain'
7
8  function App() {
9    return <SpotiMain /> // llamamos a SpotiMain
10 }
11
12 export default App
```

3. ListadoCanciones.jsx

Muestra todas las canciones en modo tabla, llamando al array de canciones, mencionado en SpotiMain.jsx

```
const ListadoCanciones = ({ canciones }) => { // Recibe datos desde el padre props
  return (
    <div>
      <h2>Listado de Canciones</h2>
      <table border="1">
        <thead>
          <tr>
            <th>Id pista</th>
            <th>Nombre</th>
            <th>Artista</th>
            <th>Album</th>
            <th>Duracion</th>
          </tr>
        </thead>
        <tbody>
          {canciones.map((cancion) => { // recorre el array
            <tr key={cancion.track_id}> {/** fila unica que se identifica con el id */}
            <td>{cancion.track_id}</td>
            <td>{cancion.track_name}</td>
            <td>{cancion.track_artist}</td>
            <td>{cancion.track_album_name}</td>
            <td>{parseInt(cancion.duration_ms / 1000)}</td> {/** para redondear 'parseInt', dividir entre 1000 pq esta en milisegundos */}
          </tr>
          </tbody>
        </table>
      </div>
    )
  }
}

export default ListadoCanciones
```

4. FiltradoCanciones.jsx

Te busca la canción según el artista

```
SpotifEx > src > components > FiltradoCanciones.jsx > FiltradoCanciones
1  import { use, useState } from "react";
2
3  const FiltradoCanciones = ({ canciones }) => {
4
5      * artista -> guarda lo escrito
6      * setArtista() -> cambia el valor
7      */
8      const [artista, setArtista] = useState("")
9      const cancionesFiltradas = canciones.filter((cancion) =>
10         cancion.track_artist.toLowerCase().includes(artista.toLowerCase())
11     )
12
13     return (
14         <div>
15             <h2>Buscar canciones por artista</h2>
16
17             <input type="text" placeholder="Escriba artista" onChange={(e) => setArtista(e.target.value)} />
18
19             <ul>
20                 {cancionesFiltradas.map((cancion) => (
21                     <li key={cancion.track_id}>
22                         {cancion.track_name}
23                     </li>
24                 ))}
25             </ul>
26         </div>
27     )
28 }
29
30
31 export default FiltradoCanciones
```

5. MasPopular.jsx

Al tardar en cargar, manda un mensaje para que no se genere error.

```
SpotifEx > src > components > MasPopular.jsx > MasPopular
1  const MasPopular = ({ canciones }) => {
2      // Comprueba si todavia no hay canciones cargadas
3      if (canciones.length === 0){
4          return <p>Cargando cancion mas popular</p>
5      }
6
7      /**
8       * reduce -> recorre el array y quedarse con un unico resultado
9       * cancion.popularity > max.popularity ? cancion : max -> quedarse con la cancion con mayor popularity
10      */
11      const masPopular = canciones.reduce((max, cancion) =>
12         cancion.track_popularity > max.track_popularity ? cancion : max
13     )
14
15     return (
16         <div>
17             <h2>Cancion mas popular</h2>
18             <p>Nombre: {masPopular.track_name}</p>
19             <p>Artista: {masPopular.track_artist}</p>
20             <p>Popularidad: {masPopular.track_popularity}</p>
21         </div>
22     )
23 }
24
25 export default MasPopular
```

ORDEN A REALIZAR UN EJERCICIO ASI

1. Crear SpotiMain
2. Mostrar SpotiMain en App.jsx
3. Cargar JSON con fetch
4. Comprobar que funciona con console.log(datos)
5. Crear tabla ListadoCanciones
6. Crear filtro FiltradoCanciones
7. Crear MasPopular

COMANDOS UTILES

useState = guardar datos

useEffect = ejecutar al cargar

props = pasar datos

map = recorrer lista

filter = filtrar lista

reduce = obtener un resultado

onChange = detectar escritura

e.target.value = texto input

fetch = cargar JSON

await = esperar

PROPS

Los props sirven para pasar datos de un componente padre a un hijo

En SpotiMain:

```
<ListadoCanciones canciones={canciones}></ListadoCanciones>
```

En ListadoCanciones:

```
const ListadoCanciones = ({ canciones }) => {
```

FILTER

filter() sirve para filtrar elementos

```
const cancionesFiltradas = canciones.filter((cancion) =>  
  cancion.track_artist.toLowerCase().includes(artist.toLowerCase()  
)
```

Se queda solo con la canción cuyo artista contiene lo escrito

RESUMEN

SpotiMain carga las canciones, las guarda en estado y se las pasa a los hijos con props.

Los hijos no cargan el JSON, solo reciben las canciones y las muestran, filtran o calculan algo.